

Chapter 30 - Processor Timing, Traps, and Exceptions

The six processors of Part IV each have their own trap and exception machinery, but they all share one bus, one clock, and one shared set of interrupt sources. This chapter describes that common ground: how time is counted, how each CPU receives an interrupt, how exceptions are raised, and how the IRQ diagnostics block at 0xF23C0 makes all of it visible.

30.1 The shared clock

Intuition Engine has a single master clock. Each processor draws from it at its own rate; instructions on the smaller CPUs cost a few cycles each, while IE64 retires one instruction per cycle in steady state. The compositor, audio mixer, and timer count cycles of the master clock independently of which CPU is active.

For BASIC programs there is no need to count cycles directly: the `WAIT` statement, the audio engines' own clocks, and the video chip's VBlank flag are usually enough. For machine code that wants precise control, each per-CPU chapter lists the published cycle counts for its instruction set; the cycle counts are accurate to the canonical manuals for the heritage chips and to the implementation for IE64 / IE32.

30.2 Timer

Every CPU sees the same 32-bit hardware timer. On IE32 and the heritage CPUs it appears as two memory-mapped registers:

Address	Name	Purpose
0xF0804	TIMER_COUNT	Live count (decrements toward 0)
0xF0808	TIMER_PERIOD	Reload value when count reaches 0

When the count reaches zero, the timer reloads from `PERIOD`. If the CPU's timer-interrupt-enable bit is set, the reload raises an interrupt on that CPU.

IE64 has the timer wired into its control register bank as well: CR9 is `TIMER_PERIOD`, CR10 is `TIMER_COUNT`, CR11 is the control word (bit 0 enable, bit 1 IRQ enable). IE32 has dedicated opcodes `SEI/CLI/RTI` that manipulate its single interrupt flag.

30.3 Interrupt sources

There are four canonical hardware interrupt sources:

- **VBlank** - raised by VideoChip at the start of each vertical blanking interval. Mapped to M68K level 5.
- **Copper / raster** - raised by the copper-list engine when a `WAIT` matches the current raster position. Mapped to M68K level 5.
- **Audio** - raised by any audio engine that has reached the end of a loop or sample. Mapped to M68K level 4.
- **Timer** - raised when the shared timer reloads.

For the heritage CPUs the system collapses these into one or two lines:

- 6502: IRQ (maskable) and NMI (non-maskable). Audio and timer share IRQ; VBlank uses NMI on platforms that need it.
- Z80: INT (maskable) and NMI. The interrupt-mode setting (IM 0/IM 1/IM 2) chooses how the vector is formed.
- M68K: seven prioritised levels. IE drives L4 for audio and L5 for video, both as auto-vectored interrupts. Level 7 is non-maskable; IE does not raise it.
- x86: a single INTR line plus NMI (vector 2). The vector number is supplied by the interrupting device.
- IE32: a single interrupt vector at memory 0x0004.
- IE64: the trap vector in CR_TRAP_VEC, or the timer-specific CR_INTR_VEC.

30.4 Exceptions versus interrupts

Across all six CPUs the distinction is the same:

- **Interrupt** - caused by an external event (a device finishing a frame, a timer expiring). Maskable in general, except for NMI and M68K level 7.
- **Exception** - caused by the executing instruction itself (divide by zero, address error, illegal opcode, privilege violation, breakpoint, syscall).

Both end the current instruction and jump to a vector through the same mechanism. The CPU's view of "what just happened" is the distinction.

30.5 Trap and exception causes

The five canonical trap causes that every CPU can raise:

Cause	6502	Z80	M68K vector	x86 vector	IE32 / IE64
Reset	0xFFFC/D	RESET pin	0/1	EIP=0	0x0000
Divide by zero	(none)	(none)	5	0	IE64 FAULT_WRITE_DENIED sibling fault
Illegal opcode	(none)	(none)	4	6	IE64 FAULT_ILLEGAL_INSTRUCTION
Privilege violation	(none)	(none)	8	13	IE64 FAULT_PRIV
Software trap	BRK	RST n	TRAP #n	INT n	IE64 SYSCALL
Breakpoint	(none)	(none)	(debugger)	INT 3	(debugger)
Bus / address error	(none)	(none)	2 / 3	(none)	(none)
Trace / single-step	(none)	(none)	9	TF flag	IE64 FAULT_TRACE (not implemented)

The smaller CPUs simply do not have the silicon to detect some of these; an illegal opcode on the 6502, for example, just executes the corresponding undocumented operation (see Chapter 26 §26.6).

30.6 The IRQ diagnostics block

The 32-byte block at 0xF23C0 reports interrupt activity in real time. Programs do not normally need it; it is here for debugging stuck handlers.

Address	Name	Reports
0xF23C0	IRQ_DIAG_ISR	Interrupt-in-service bitmask

Address	Name	Reports
0xF23C4	IRQ_DIAG_FLAGS	b0 stopped, b1 in-exception, b2 INTENA, b3 running
0xF23C8	IRQ_DIAG_PENDING	Pending-interrupt bitmask
0xF23CC	IRQ_DIAG_COUNTERS	L5 delivered (lo16) + L4 delivered (hi16)
0xF23D0	IRQ_DIAG_BLOCKED	L5 blocked (lo16) + L4 blocked (hi16)
0xF23D4	IRQ_DIAG_RTE	RTE count
0xF23D8	IRQ_DIAG_STOP_SPINS	Consecutive STOP iterations without wake
0xF23DC	IRQ_DIAG_WATCHDOG	Latched watchdog event count

The counters are most useful when the M68K is the active CPU. A non-zero IRQ_DIAG_BLOCKED value plus a stable STOP_SPINS counter is a classic sign of an interrupt-mask leak: the device raised the IRQ, the CPU was running with the mask too high, and the IRQ was deferred indefinitely.

30.7 Reset

Reset is the universal exception: every CPU has one, and it always runs to completion before any other code executes.

- **IE64** - PC = 0x0000, all GPRs zeroed, MMU disabled, mode = supervisor. The reset vector is the first instruction of the loaded image; convention is JMP PROG_START.
- **IE32** - PC = 0x0000, SP = STACK_START (0x9F000), interrupts disabled. Convention is JMP 0x1000.
- **6502** - PC is loaded from 0xFFFF/0xFFFD, I flag set.
- **Z80** - PC = 0x0000, interrupt mode set to 0, IFF1 and IFF2 cleared.
- **M68K** - SSP loaded from vector 0, PC loaded from vector 1. Supervisor mode entered. I2:I1:I0 = 7 (all interrupts masked).
- **x86** - EIP = 0x00000000, all segments cleared to 0, IF = 1.

On Intuition Engine the boot sequence loads the program executable into low RAM and then issues a reset on the chosen CPU, which makes the program's first instruction the effective reset vector. Chapter 31 covers the executor in detail.

30.8 Timing on the heritage CPUs

The heritage chips have well-known cycle counts for every instruction; their canonical manuals are the authority. Intuition Engine's cycle counters tick at one master-clock tick per CPU cycle, so an instruction documented as 4 cycles consumes four ticks of the timer.

The smaller CPUs run at the speed of the master clock divided by a per-CPU scaling factor: the 6502 and Z80 each get roughly the same instruction throughput as a real 4-MHz part on a busy period, but the master clock continues at the full IE rate so a 6502 instruction does not stretch the VBlank interval. Programs that need very precise timing should use the timer or the copper-list rather than counting instructions.

IE64's cycle counter is reported in CR10 (the timer count) and the implementation increments it once per retired instruction. The published latencies in Chapter 24 are minimum bounds; modern implementations may retire instructions faster in straight-line code.

30.9 What comes next

Chapter 31 covers how programs running on different CPUs cooperate through the coprocessor and cross-CPU calling mechanism, plus the C0START/C0STOP/C0WAIT BASIC verbs that drive it. After that, Chapters 32 and 33 cover the two interactive debugging surfaces: the Machine Monitor and IE Script.